$\frac{1}{1+\exp(-x)}$ or $\tanh$ are used as activation functions. $\odot$ denotes the point-wise multiplication of two vectors. Without peepholes, the LSTM memory cell forward pass rules are (see Figure A10):

$$\boldsymbol{z}^t = g\left(\boldsymbol{W_z}\,\boldsymbol{x}^t + \boldsymbol{b_z}\right) \quad \text{cell input} \tag{A326}$$

$$\boldsymbol{i}^t = \sigma\left(\boldsymbol{W_i}\,\boldsymbol{x}^t + \boldsymbol{b_i}\right) \quad \text{input gate} \tag{A327}$$

$$\boldsymbol{f}^t = \sigma\left(\boldsymbol{W_f}\,\boldsymbol{x}^t + \boldsymbol{b_f}\right) \quad \text{forget gate} \tag{A328}$$

$$\boldsymbol{c}^t = \boldsymbol{i}^t \odot \boldsymbol{z}^t + \boldsymbol{f}^t \odot \boldsymbol{c}^{t-1} \quad \text{cell state} \tag{A329}$$

$$\boldsymbol{o}^t = \sigma\left(\boldsymbol{W_o}\,\boldsymbol{x}^t + \boldsymbol{b_o}\right) \quad \text{output gate} \tag{A330}$$

$$\boldsymbol{y}^t = \boldsymbol{o}^t \odot h\left(\boldsymbol{c}^t\right) \quad \text{cell output} \tag{A331}$$

A8.3 Long-Term Dependencies vs. Uniform Credit Assignment

The LSTM network has been proposed with the aim to learn *long-term dependencies* in sequences which span over long intervals [55, 56, 50, 51]. However, besides extracting long-term dependencies, LSTM memory cells have another, even more important, advantage in sequence learning: as already described in the early 1990s, LSTM memory cells allow for *uniform credit assignment*, that is, the propagation of errors back to inputs without scaling them [49]. For uniform credit assignment of current LSTM architectures, the forget gate $\boldsymbol{f}$ must be one or close to one. A memory cell without an input gate $\boldsymbol{i}$ just sums up all the squashed inputs it receives during scanning the input sequence. Thus, such a memory cell is equivalent to a unit that sees all sequence elements at the same time, as has been shown via the "Ersatzschaltbild" [49]. If an output error occurs only at the end of the sequence, such a memory cell, via backpropagation, supplies the same delta error at the cell input unit $\boldsymbol{z}$ at every time step. Thus, all inputs obtain the same credit for producing the correct output and are treated on an equal level and, consequently, the incoming weights to a memory cell are adjusted by using the same delta error at the input unit $\boldsymbol{z}$.

In contrast to LSTM memory cells, standard recurrent networks scale the delta error and assign different credit to different inputs. The more recent the input, the more credit it obtains. The first inputs of the sequence are hidden from the final states of the recurrent network. In many learning tasks, however, important information is distributed over the entire length of the sequence and can even occur at the very beginning. For example, in language- and text-related tasks, the first words are often important for the meaning of a sentence. If the credit assignment is not uniform along the input sequence, then learning is very limited. Learning would start by trying to improve the prediction solely by using the most recent inputs. Therefore, the solutions that can be found are restricted to those that can be constructed if the last inputs are considered first. Thus, only those solutions are found that are accessible by gradient descent from regions in the parameter space that only use the most recent input information. In general, these limitations lead to sub-optimal solutions, since learning gets trapped in local optima. Typically, these local optima correspond to solutions which efficiently exploit the most recent information in the input sequence, while information way back in the past is neglected.

A8.4 Special LSTM Architectures for contribution Analysis

A8.4.1 LSTM for Integrated Gradients

For Integrated Gradients contribution analysis with LSTM, we make following assumptions:

- **(A1)** $\boldsymbol{f}^t = 1$ for all t. That is the forget gate is always 1 and nothing is forgotten. We assume uniform credit assignment, which is ensured by the forget gate set to one.
- **(A2)** $\boldsymbol{o}^t = 1$ for all t. That is the output gate is always 1 and nothing is forgotten.
- **(A3)** We set $h = a_h \tanh$ with $a_h = 1, 2, 4$.
- **(A4)** We set $g = a_g \tanh$ with $a_g = 1, 2, 4$.
- **(A5)** The cell input gate $\boldsymbol{z}$ is only connected to the input but not to other memory cells. $\boldsymbol{W_z}$ has only connections to the input.
- **(A6)** The input gate $\boldsymbol{i}$ is not connected to the input, that is, $\boldsymbol{W_i}$ has only connections to other memory cells. This ensures that LRP assigns relevance only via $\boldsymbol{z}$ to the input.
- **(A7)** The input gate $\boldsymbol{i}$ has a negative bias, that is, $\boldsymbol{b_i} < 0$. The negative bias reduces the drift effect, that is, the memory content $\boldsymbol{c}$ either increases or decreases over time. Typical values are $\boldsymbol{b_i} = -1, -2, -3, -4, -5$.

(A8) The memory cell content is initialized with zero at time $t = 0$, that is, $\boldsymbol{c}^0 = 0$.

The resulting LSTM forward pass rules for Integrated Gradients are:

$$\boldsymbol{z}^t = a_g \, \sigma\left(\boldsymbol{W_z} \, \boldsymbol{x}^t + \boldsymbol{b_z}\right) \qquad \text{cell input} \qquad \text{(A332)}$$

$$\boldsymbol{i}^t = \sigma\left(\boldsymbol{W_i} \, \boldsymbol{x}^t + \boldsymbol{b_i}\right) \qquad \text{input gate} \qquad \text{(A333)}$$

$$\boldsymbol{c}^t = \boldsymbol{i}^t \odot \boldsymbol{z}^t + \boldsymbol{c}^{t-1} \qquad \text{cell state} \qquad \text{(A334)}$$

$$\boldsymbol{y}^t = a_h \, \tanh\left(\boldsymbol{c}^t\right) \qquad \text{cell output} \qquad \text{(A335)}$$

See Figure A11 which depicts these forward pass rules for Integrated Gradients.

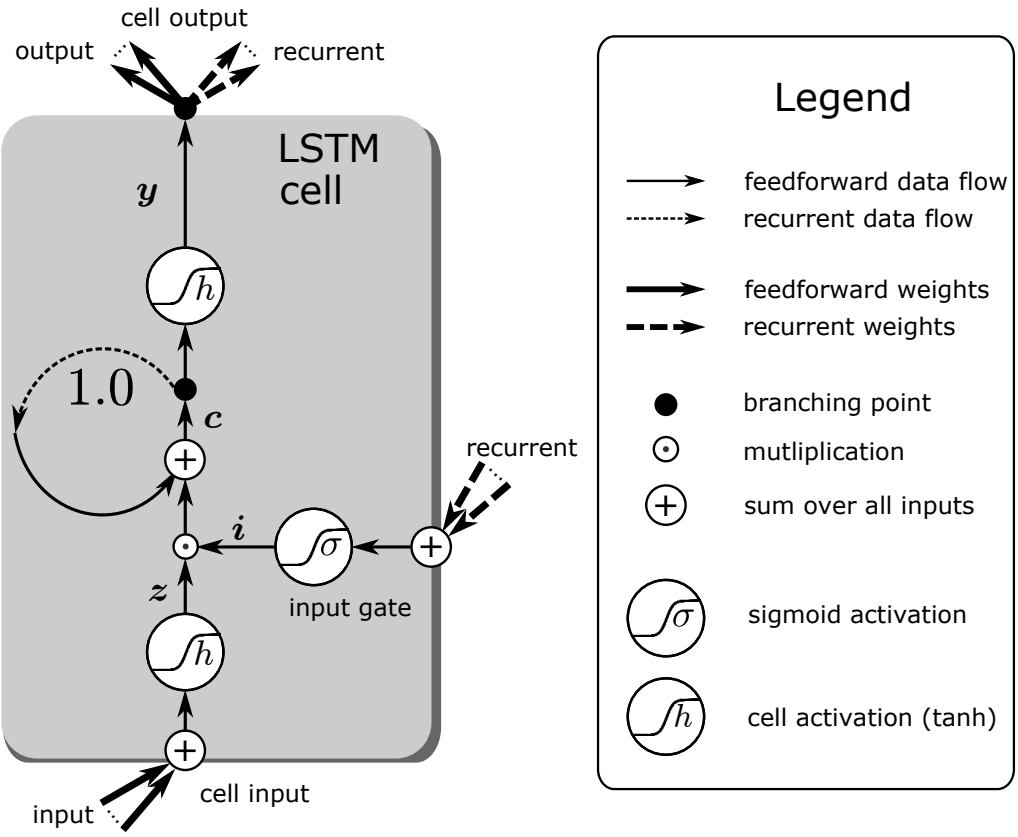

Figure A11: LSTM memory cell used for Integrated Gradients (IG). Forget gates and output gates are set to 1 since they can modify all cell inputs at times after they have been observed, which can make the dynamics highly nonlinear.

A8.4.2 LSTM for LRP

LRP has already been used for LSTM in order to identify important terms in sentiment analysis [1]. In texts, positive and negative terms with respect to the topic could be identified.

For LRP contribution analysis with LSTM, we make following assumptions:

(A1) $\boldsymbol{f}^t = 1$ for all t. That is the forget gate is always 1 and nothing is forgotten. We assume uniform credit assignment, which is ensured by the forget gate set to one.

(A2) $g > 0$, that is, g is positive. For example we can use a sigmoid $\sigma(x) = a_g \frac{1}{1+\exp(-x)}$: $g(x) = a_g \sigma(x)$, with $a_g = 2, 3, 4$. Methods like LRP have problems with negative contributions which cancel with positive contributions [84]. With a positive g all contributions are positive. The cell input $\boldsymbol{z}$ (the function g) has a negative bias, that is, $\boldsymbol{b_z} < 0$. This is important to avoid the drift effect. The drift effect is that the memory content only gets positive contributions which lead to an increase of $\boldsymbol{c}$ over time. Typical values are $\boldsymbol{b_z} = -1, -2, -3, -4, -5$.

(A3) We want to ensure that $h(0) = 0$. If the memory content is zero then nothing is transferred to the next layer. Therefore we set $h = a_h \tanh$ with $a_h = 1, 2, 4$.

(A4) The cell input gate $\boldsymbol{z}$ is only connected to the input but not to other memory cells. $\boldsymbol{W_z}$ has only connections to the input. This ensures that LRP assigns relevance $\boldsymbol{z}$ to the input and $\boldsymbol{z}$ is not disturbed by redistributing relevance to the network.

(A5) The input gate $\boldsymbol{i}$ is not connected to the input, that is, $\boldsymbol{W_i}$ has only connections to other memory cells. This ensures that LRP assigns relevance only via $\boldsymbol{z}$ to the input.

(A6) The output gate $\boldsymbol{o}$ is not connected to the input, that is, $\boldsymbol{W_o}$ has only connections to other memory cells. This ensures that LRP assigns relevance only via $\boldsymbol{z}$ to the input.

(A7) The input gate $\boldsymbol{i}$ has a negative bias, that is, $\boldsymbol{b_i} < 0$. Like with the cell input the negative bias avoids the drift effect. Typical values are $\boldsymbol{b_i} = -1, -2, -3, -4$.

(A8) The output gate $\boldsymbol{o}$ may also have a negative bias, that is, $\boldsymbol{b_o} < 0$. This allows to bring in different memory cells at different time points. It is related to resource allocation.

(A9) The memory cell content is initialized with zero at time $t = 0$, that is, $\boldsymbol{c}^0 = 0$. The memory cell content $\boldsymbol{c}^t$ is non-negative $\boldsymbol{c}^t \geqslant 0$ since $\boldsymbol{z} \geqslant 0$ and $\boldsymbol{i} \geqslant 0$.

The resulting LSTM forward pass rules for LRP are:

$$\boldsymbol{z}^t \;=\; a_g\, \sigma\left(\boldsymbol{W_z}\, \boldsymbol{x}^t \;+\; \boldsymbol{b_z}\right) \qquad \text{cell input} \tag{A336}$$

$$\boldsymbol{i}^t \;=\; \sigma\left(\boldsymbol{W_i}\, \boldsymbol{x}^t \;+\; \boldsymbol{b_i}\right) \qquad \text{input gate} \tag{A337}$$

$$\boldsymbol{c}^t \;=\; \boldsymbol{i}^t \odot \boldsymbol{z}^t \;+\; \boldsymbol{c}^{t-1} \qquad \text{cell state} \tag{A338}$$

$$\boldsymbol{o}^t \;=\; \sigma\left(\boldsymbol{W_o}\, \boldsymbol{x}^t \;+\; \boldsymbol{b_o}\right) \qquad \text{output gate} \tag{A339}$$

$$\boldsymbol{y}^t \;=\; \boldsymbol{o}^t \odot a_h\, \tanh\left(\boldsymbol{c}^t\right) \qquad \text{cell output} \tag{A340}$$

See Figure A12 which depicts these forward pass rules for LRP. However, gates may be used while no relevance is given to them which may lead to inconsistencies.

LRP and Contribution Propagation for LSTM. We analyze Layer-wise Relevance Propagation (LRP) and Contribution Propagation for LSTM networks. A single memory cell can be described by:

$$c^t \;=\; i^t\, z^t \;+\; c^{t-1}\,. \tag{A341}$$

Here we treat i^t like a weight for z^t and c^{t-1} has weight 1.
For positive values of i^t, z^t, and c^{t-1}, both LRP and contribution propagation leads to

$$R_{c^t \leftarrow y^t} \;=\; R_{y^t} \tag{A342}$$

$$R_{c^t} \;=\; R_{c^t \leftarrow c^{t+1}} \;+\; R_{c^t \leftarrow y^t} \tag{A343}$$

$$R_{c^{t-1} \leftarrow c^t} \;=\; \frac{c^{t-1}}{c^t}\, R_{c^t} \tag{A344}$$

$$R_{z^t \leftarrow c^t} \;=\; \frac{i^t\, z^t}{c^t}\, R_{c^t}\,. \tag{A345}$$

Since we predict only at the last step $t = T$, we have $R_{y^t} = 0$ for $t < T$. For $t = T$ we obtain $R_{c^T} = R_{y^T}$, since $R_{c^T \leftarrow c^{T+1}} = 0$.
We obtain for $t = 1 \ldots T$:

$$R_{c^T} \;=\; R_{y^T} \tag{A346}$$

$$R_{c^{t-1}} \;=\; \frac{c^{t-1}}{c^t}\, R_{c^t} \tag{A347}$$

which gives

$$R_{c^t} \;=\; R_{y^T} \prod_{\tau=t+1}^{T} \frac{c^{\tau-1}}{c^\tau} \;=\; \frac{c^t}{c^T}\, R_{y^T} \tag{A348}$$

and consequently as $c^0 = 0$ we obtain

$$R_{c^0} \;=\; 0\,, \tag{A349}$$

$$R_{z^t} \;=\; \frac{i^t\, z^t}{c^T}\, R_{y^T}\,. \tag{A350}$$